DocuMind: Accurate Document Question Answering using Semantic Retrieval and RAG

Abstract:
Retrieval-Augmented Generation (RAG) addresses large language model hallucination by grounding model responses in verified external document collections. In this paper, we propose DocuMind, a modular architecture combining robust PDF text preprocessing, dense semantic vector retrieval via FAISS, and citation-grounded generation.

1. Introduction
Natural Language Processing has witnessed significant breakthroughs with Transformer architectures. However, applying standard generative language models directly to domain-specific documents presents two critical challenges: factual hallucination and the lack of exact citation references. Standard search engines rely on lexical keyword matching such as BM25 or TF-IDF, which fail when users formulate queries using synonyms or conceptual questions not explicitly containing query keywords. Dense retrieval maps queries and passages into a shared vector embedding space where semantic similarity is measured by cosine similarity or inner product.

DocuMind resolves these limitations by executing a four-stage sequential pipeline: First, documents are parsed page-by-page, normalizing Unicode representations and tracking 1-based page metadata. Second, text is partitioned into overlapping windows preserving sentence boundaries. Third, dense vector embeddings are indexed using FAISS IndexFlatIP. Fourth, relevant passages are passed with strict attribution constraints to the generation model.

2. Document Preprocessing and Ingestion Pipeline

The first stage in RAG systems is document ingestion. Unstructured PDF files present unique challenges including non-standard font encodings, hyphenated line wraps, non-breaking spaces, and varying page layouts.

2.1 Text Extraction and Normalization

We employ PyMuPDF (fitz) to extract text content in reading order. Raw extracted text undergoes canonical Unicode NFC normalization, non-breaking space replacement, and removal of unprintable control characters. Unlike lossy cleaners that strip mathematical symbols and punctuation, our cleaner conservatively preserves punctuation marks, parentheses, and technical symbols essential for semantic understanding.

2.2 Page-Constrained Chunking

Chunking strategies greatly influence downstream retrieval accuracy. Overly large chunks dilute specific semantic signals, while overly small chunks fragment conceptual coherence. DocuMind implements a paragraph- and sentence-aware sliding window with a target size of 1,200 characters and an overlap of 180 characters. Critically, chunks never cross page boundaries. This design decision ensures that every citation presented to end users points unambiguously to a single verified page number in the source document.

3. Semantic Vector Search and FAISS Indexing

3.1 Embedding Model

Extracted chunks are transformed into 384-dimensional dense vectors using the SentenceTransformer 'all-MiniLM-L6-v2' model. MiniLM offers an optimal trade-off between inference speed and representation quality, making it suitable for real-time document search on consumer CPUs.

3.2 Cosine Similarity through L2 Normalization and Inner Product

Cosine similarity measures the cosine of the angle between two multi-dimensional vectors. By performing L2 normalization on both chunk embeddings and query embeddings prior to indexing, the denominator in the cosine similarity formula simplifies to 1. Consequently, computing cosine similarity reduces to an inner product (dot product). In FAISS, this is implemented using IndexFlatIP, leveraging AVX-256 SIMD parallel instructions for ultra-fast nearest-neighbor retrieval without needing clustering approximation.

3.3 Baseline Comparison

For empirical evaluation, DocuMind maintains a classical TF-IDF baseline with scikit-learn. Experimental runs demonstrate that while TF-IDF excels on exact technical term lookup, dense semantic search achieves significantly higher recall on conceptual queries and natural language paraphrases.